

Analiza experimentală comparativă a algoritmilor de sortare

Influența structurii de date, distribuției inputului și implementării asupra
performanței

Butoi Alexandru-Gabriel
Universitatea de Vest din Timișoara
Facultatea de Informatică
Specializarea Informatică, Română
[alexandru.butoi06@e-uvv.ro]

2026

Rezumat

Lucrarea prezintă o analiză experimentală comparativă a mai multor algoritmi de sortare, testați pe structuri de date diferite: **array**, **vector** și **listă simplu înlănțuită**. Problema cercetată este alegerea unei metode eficiente de sortare în condiții practice, unde performanța nu depinde doar de complexitatea asimptotică, ci și de localitatea în memorie, distribuția datelor, costurile auxiliare și calitatea implementării. Rezultatele confirmă diferența majoră dintre algoritmi de complexitate $O(n^2)$ și cei de complexitate $O(n \log n)$, dar arată și că algoritmi teoretic apropiați pot avea timpi reali foarte diferiți. Lucrarea include formularea problemei, analiza audienței, obiective operaționale, cadrul metodologic, rezultatele experimentale, comparația cu literatura de specialitate, limitele cercetării și direcțiile viitoare.

Cuvinte-cheie: algoritmi de sortare, complexitate, QuickSort, MergeSort, IntroSort, structuri de date, localitate în memorie, analiză experimentală.

Cuprins

1	Introducere	3
1.1	Problema cercetată	3
1.2	Demersul lucrării	3
2	Analiza audienței și obiective operaționale	3
2.1	Obiective operaționale	4
2.2	Motivație, învățare și asociere	4
3	Literatura de specialitate și rolul surselor	4

4	Cadrul formal al problemei	5
4.1	Criterii teoretice	5
4.2	Ipoteze de cercetare	6
5	Descrierea soluțiilor existente și a algoritmilor analizați	6
5.1	Algoritmi elementari	6
5.2	Algoritmi eficienți de tip divide et impera	7
5.3	Algoritmi hibridi și implementări standard	7
6	Metodologia cercetării	7
6.1	Mediul de execuție	7
6.2	Seturi de date și măsurare	8
6.3	Validitatea măsurărilor	8
7	Modelare și implementare	8
7.1	Structuri de date analizate	8
7.2	Criterii de evaluare	8
7.3	Normalizarea datelor	9
7.4	Modelul de cost urmărit experimental	9
7.5	Reproductibilitatea experimentului	9
8	Rezultate experimentale și interpretare	9
8.1	Algoritmi de complexitate $O(n^2)$	9
8.2	Algoritmi de complexitate $O(n \log n)$	10
8.3	Influența structurii de date	10
8.4	Merge Sort paralel și QuickSort	11
8.5	IntroSort manual și <code>std::sort</code>	11
8.6	Sensibilitatea la distribuția datelor	12
8.7	Interpretare sintetică a rezultatelor	12
9	Comparație cu literatura de specialitate	13
10	Limite ale cercetării	13
11	Direcții viitoare de cercetare	14
12	Transpunerea rezultatelor în forma lucrării	14
13	Concluzii finale	15

1. Introducere

Sortarea este una dintre problemele fundamentale ale informaticii, având aplicații în procesarea datelor, baze de date, sisteme de fișiere, căutare, analiză numerică și aplicații software de scară mare. Deși algoritmi de sortare sunt frecvent prezentați prin complexitatea lor asimptotică, performanța reală a unei implementări depinde de factori care nu sunt surprinși complet de notațiile $O(n^2)$ sau $O(n \log n)$.

În literatura de specialitate, algoritmi sunt analizați după complexitate temporală, complexitate spațială, stabilitate și adaptabilitate la input [2, 5, 7]. Totuși, într-un program real, aceeași clasă teoretică poate produce timpi foarte diferiți. Intervin localitatea în memorie, numărul de copii, costurile recursiei, strategiile de partiționare, tipul de structură de date și distribuția inputului.

1.1. Problema cercetată

Problema centrală a lucrării este următoarea:

În ce măsură alegerea algoritmului de sortare, a structurii de date și a implementării concrete influențează timpul real de execuție?

Lucrarea pornește de la ideea că analiza teoretică este necesară, dar nu suficientă. Un algoritm poate fi corect și bine cunoscut, însă poate deveni impracticabil pentru volume mari de date. De aceea, studiul urmărește compararea comportamentului practic al mai multor algoritmi de sortare pe aceleași seturi de date și în același mediu de execuție.

1.2. Demersul lucrării

Lucrarea urmează un demers specific unei cercetări aplicate:

1. **înțelegerea problemei:** identificarea sortării ca problemă fundamentală și stabilirea factorilor care influențează performanța practică;
2. **căutarea în literatura de specialitate:** consultarea lucrărilor clasice și a surselor tehnice relevante;
3. **evaluarea soluțiilor existente:** analiza algoritmilor consacrați și a implementărilor standard, în special `std::sort`;
4. **elaborarea unei soluții proprii:** construirea unui cadru experimental de testare și interpretarea rezultatelor obținute.

Conținutul lucrării este organizat în trei părți: partea formală, partea de modelare și implementare, respectiv partea experimentală. Închiderea include comparația cu literatura de specialitate, concluziile și direcțiile viitoare de cercetare.

2. Analiza audienței și obiective operaționale

Lucrarea se adresează unui public cu nivel mediu și avansat de cunoștințe în informatică, în special studenților care au parcurs noțiuni de programare, structuri de date și complexitate

algoritmică. Publicul țintă trebuie să cunoască noțiuni precum recursivitate, alocare dinamică, pointeri, structuri liniare și diferența dintre analiză teoretică și măsurare experimentală.

Nivelul de interes al audienței este orientat spre înțelegerea diferenței dintre performanța teoretică și performanța practică. Într-un curs de algoritmă, este simplu să se afirme că un algoritm este $O(n^2)$, iar altul este $O(n \log n)$. În practică, aceste diferențe se reflectă în secunde, minute, ore sau chiar estimări de zile pentru rulări mari.

Așteptările audienței sunt: explicații clare, rezultate experimentale relevante, comparații între structuri de date, concluzii aplicabile în programare și o legătură explicită între teorie, implementare și performanță.

2.1. Obiective operaționale

La finalul parcurgerii lucrării, cititorul trebuie să fie capabil să:

1. explice diferența dintre complexitatea teoretică și timpul real de execuție;
2. compare algoritmi de sortare din clase diferite de complexitate;
3. identifice impactul structurii de date asupra performanței;
4. interpreteze rezultatele experimentale în raport cu literatura de specialitate;
5. justifice alegerea unui algoritm de sortare într-un context concret;
6. propună direcții de optimizare sau extindere a experimentului.

2.2. Motivație, învățare și asociere

Motivația lucrării pornește de la o realitate practică: alegerea algoritmului potrivit poate face diferența dintre o problemă rezolvabilă în câteva secunde și una imposibil de rulat într-un timp acceptabil. De aceea, lucrarea aduce cititorul pe subiect printr-o întrebare aplicată: de ce un algoritm corect teoretic poate deveni inutilizabil pe date mari?

Învățarea urmărită nu constă doar în memorarea complexităților, ci în asocierea lor cu rezultate experimentale concrete. Cititorul observă cum aceeași problemă, sortarea, se comportă diferit în funcție de algoritm, structură de date și distribuția inputului. Asocierea apare atunci când rezultatele confirmă utilitatea practică a teoriei: ceea ce a fost învățat permite rezolvarea unor probleme mai mari, mai rapid și cu resurse mai bine controlate.

3. Literatura de specialitate și rolul surselor

Bibliografia utilizată are două roluri: fundamentarea teoretică a algoritmilor și atribuirea corectă a contribuțiilor originale. Nu este suficientă citarea unei singure surse generale pentru toți algoritmii. Cormen et al. [2], Knuth [5] și Sedgewick și Wayne [7] sunt surse potrivite pentru analiza generală, dar algoritmii specifici trebuie creditați prin lucrările relevante.

QuickSort este asociat cu lucrarea originală a lui Hoare [4]. HeapSort este creditat prin Williams [8], iar IntroSort prin Musser [6]. Pentru problema implementărilor eficiente de sortare, lucrarea lui Bentley și McIlroy este relevantă deoarece arată că performanța unei funcții de sortare depinde decisiv de detaliile de implementare [1]. Documentația `std::sort` este folosită ca sursă tehnică pentru implementările standard moderne [3].

Algoritm / con- cept	Sursă	Rol în lucrare
Bubble, Selection, Insertion	[2, 5, 7]	Fundamentare didactică și complexitate.
Merge Sort	[5, 2]	Metoda divide et impera și interclasarea.
QuickSort	[4]	Creditarea algoritmului original.
HeapSort	[8]	Baza componentei HeapSort din IntroSort.
IntroSort	[6]	Algoritm hibrid introspectiv.
Implementări eficiente	[1, 3]	Optimizări practice și etalon tehnic.
Rezultate experimentale proprii	Rapoartele și măsurătorile generate în cadrul lucrării	Sursa primară internă pentru grafice și interpretări experimentale.

Tabela 1: Rolul surselor bibliografice utilizate în lucrare.

Sursele nu sunt folosite doar formal, ci metodologic: unele explică teoria, altele oferă credit istoric, iar rezultatele experimentale proprii susțin partea aplicativă a lucrării. Aceste rezultate nu sunt introduse în bibliografie ca sursă externă, deoarece reprezintă contribuția autorului, descrisă în metodologia experimentului și în secțiunea de rezultate.

4. Cadrul formal al problemei

Problema sortării poate fi formulată astfel: pentru un șir de valori $a_1, a_2, \dots, a_n$, se cere obținerea unei permutări a elementelor astfel încât $a_1 \leq a_2 \leq \dots \leq a_n$. În această lucrare, criteriul de ordonare este ordinea numerică naturală, iar accentul cade pe timpul de execuție necesar obținerii rezultatului.

4.1. Criterii teoretice

Din punct de vedere teoretic, algoritmii sunt comparați prin:

- complexitate temporală în caz favorabil, mediu și nefavorabil;
- complexitate spațială;
- stabilitate;
- caracter in-place sau cu memorie auxiliară;
- sensibilitate la distribuția inputului.

Această clasificare permite o primă diferențiere între metode. Algoritmii quadratici, precum Bubble Sort sau Selection Sort, au valoare didactică, dar devin rapid impracticabili. Algoritmii de tip $O(n \log n)$, precum Merge Sort, QuickSort și IntroSort, sunt mult mai potriviți pentru volume mari de date.

Algoritm	Complexitate medie	Memorie auxiliară	Stabilitate	Observație practică
Bubble Sort	$O(n^2)$	$O(1)$	stabil	util doar didactic sau pe date foarte mici
Selection Sort	$O(n^2)$	$O(1)$	de regulă instabil	puține schimburi, dar multe comparații
Binary Insertion Sort	$O(n^2)$	$O(1)$	stabil	reduce comparațiile, nu și deplasările
Merge Sort	$O(n \log n)$	$O(n)$	stabil	predictibil, dar cu cost de copiere
QuickSort	$O(n \log n)$ mediu	$O(\log n)$	instabil	foarte eficient în place
IntroSort	$O(n \log n)$	$O(\log n)$	instabil	robust prin strategie hibridă

Tabela 2: Caracteristici teoretice generale ale algoritmilor analizați.

4.2. Ipoteze de cercetare

Au fost urmărite următoarele ipoteze:

1. algoritmii $O(n^2)$ devin impracticabili pentru volume mari de date;
2. structurile contigue, precum **array** și **vector**, sunt mai eficiente decât lista simplu înlanțuită;
3. QuickSort cu pivot random are performanțe foarte bune în practică;
4. paralelizarea nu garantează automat o îmbunătățire a timpului de execuție;
5. implementările standard optimizate pot depăși semnificativ variantele manuale.

5. Descrierea soluțiilor existente și a algoritmilor analizați

Pentru ca experimentul să fie interpretat corect, algoritmii analizați trebuie priviți nu doar ca formule de complexitate, ci ca soluții diferite pentru aceeași problemă. Fiecare metodă are un mecanism propriu de lucru, iar acest mecanism explică de ce performanțele diferă în practică.

5.1. Algoritmi elementari

Bubble Sort, Selection Sort și Binary Insertion Sort sunt algoritmi potriviți pentru înțelegerea mecanismelor de bază ale sortării. Bubble Sort compară elemente vecine și mută progresiv valorile mari spre finalul șirului. Selection Sort caută repetat minimul din zona nesortată și îl așază pe poziția corectă. Binary Insertion Sort îmbunătățește varianta clasică de insertion sort prin folosirea căutării binare pentru identificarea poziției de inserare.

Aceste soluții au valoare didactică deoarece arată direct relația dintre numărul de comparații, numărul de mutări și creșterea dimensiunii inputului. Totuși, ele nu sunt soluții potrivite

pentru aplicații mari. Chiar dacă unele variante reduc numărul de comparații, mutarea elementelor păstrează costul total în zona quadratică. În experiment, aceste metode au fost incluse tocmai pentru a oferi un termen de comparație clar față de algoritmi eficienți.

5.2. Algoritmi eficienți de tip divide et impera

Merge Sort și QuickSort aparțin familiei de algoritmi divide et impera, dar folosesc strategii diferite. Merge Sort împarte datele în două părți, sortează recursiv fiecare parte și apoi interclasează rezultatele. Avantajul său major este predictibilitatea, iar dezavantajul este memoria auxiliară folosită pentru interclasare.

QuickSort, în schimb, alege un pivot și rearanjează elementele astfel încât valorile mai mici să ajungă într-o parte, iar cele mai mari în cealaltă. Avantajul său practic este că lucrează in-place și are constante mici de execuție. Dezavantajul teoretic este existența cazului nefavorabil, însă alegerea random a pivotului reduce semnificativ probabilitatea acestuia în testele obișnuite.

5.3. Algoritmi hibridi și implementări standard

IntroSort este important deoarece arată cum soluțiile moderne nu se limitează la un singur algoritm pur. Strategia introspectivă pornește de la QuickSort, monitorizează adâncimea recursiei și trece la HeapSort când există risc de degradare. Pentru subsecvențe mici, implementările performante folosesc de obicei variante de Insertion Sort, deoarece overhead-ul recursiei devine mai costisitor decât sortarea directă.

Această idee explică de ce `std::sort` este un etalon practic: nu este doar o transcriere simplă a unui algoritm din manual, ci o implementare rafinată, cu praguri, optimizări și decizii de inginerie testate pe multe cazuri. Din acest motiv, comparația cu `std::sort` are rolul de a arăta diferența dintre o implementare didactică și una industrială.

6. Metodologia cercetării

Pentru fiecare algoritm au fost colectați timpi de execuție pe același set de teste. S-a urmărit separat comportamentul pe `array`, `vector` și listă simplă înlănțuită. Această abordare separă două surse de variație: variația datorată algoritmului și variația datorată structurii de date.

6.1. Mediul de execuție

Testele au fost rulate într-un mediu hardware și software controlat:

- sistem de operare: Windows 11 Pro;
- mediu de execuție: Windows Subsystem for Linux, utilizat prin Visual Studio Code;
- procesor: Intel Core i7-13650HX;
- memorie RAM: 16 GB, 4800 MT/s;
- placă video: NVIDIA GeForce RTX 4060 Laptop GPU.

Aceeași configurație a fost folosită pentru toate testele. Valorile absolute pot depinde de platformă, dar comparațiile relative dintre algoritmi sunt relevante în cadrul acestui studiu.

Nu au fost aplicate optimizări manuale de sistem, precum overclocking, limitare de frecvență sau control explicit al afinității thread-urilor.

6.2. Seturi de date și măsurare

Seturile de date au acoperit cazuri mici, medii și foarte mari, până la ordinul sutelor de milioane de elemente. Au fost utilizate distribuții random, sortate, invers sortate, egale, aproape sortate, cu duplicate sau cu interval numeric restrâns.

Timpul măsurat reprezintă timpul efectiv al sortării. Citirea datelor s-a realizat înainte de pornirea cronometrării, iar scrierea rezultatului după oprirea acesteia. Astfel, timpul de I/O nu este amestecat cu timpul algoritmului de sortare propriu-zis. Această separare este importantă deoarece altfel rezultatul ar măsura și performanța discului, nu doar eficiența algoritmului.

6.3. Validitatea măsurărilor

Pentru ca rezultatele să fie comparabile, aceeași metodă de măsurare a fost aplicată tuturor algoritmilor. Totuși, lucrarea nu pretinde că valorile obținute sunt universale. Ele sunt relevante pentru mediul de testare specificat. Scopul principal nu este stabilirea unui timp absolut valabil pe orice sistem, ci observarea diferențelor relative dintre algoritmi și structuri de date.

7. Modelare și implementare

Au fost analizate următoarele familii de algoritmi: Bubble Sort, Selection Sort, Binary Insertion Sort, Merge Sort clasic, Merge Sort paralel, QuickSort și IntroSort. Alegerea lor acoperă atât metode elementare de complexitate $O(n^2)$, cât și metode eficiente de complexitate $O(n \log n)$.

7.1. Structuri de date analizate

Structurile de date testate au fost:

- **array**, cu stocare contiguă și acces direct;
- **vector**, cu stocare contiguă, dar cu mecanisme suplimentare de gestionare;
- listă simplu înlănțuită, cu acces secvențial și noduri legate prin pointeri.

Compararea acestor structuri este importantă deoarece performanța sortării nu depinde doar de algoritm, ci și de modul în care datele sunt așezate în memorie. **Array**-ul și **vector**-ul beneficiază de localitate spațială, în timp ce lista este penalizată de dereferențieri repetate și acces secvențial.

7.2. Criterii de evaluare

Evaluarea s-a realizat pe baza timpului de execuție, a comportamentului pe volume mari de date, a sensibilității la distribuția inputului, a costurilor auxiliare de memorie și a diferențelor dintre implementări manuale și implementări standard. Pentru algoritmi paraleli a fost urmărit și raportul dintre costul sincronizării și beneficiul împărțirii lucrului pe thread-uri.

7.3. Normalizarea datelor

O direcție secundară analizată conceptual este normalizarea tipurilor de date. Valorile reale pot fi transformate în întregi prin scalare cu 10^p , unde p este numărul de zecimale controlate. Caracterele pot fi interpretate direct prin codurile lor numerice. Această idee poate simplifica infrastructura de testare, dar trebuie aplicată atent, deoarece poate introduce overflow sau pierdere de informație.

7.4. Modelul de cost urmărit experimental

În analiza rezultatelor, timpul total de sortare a fost interpretat ca rezultat al mai multor costuri parțiale. Primul cost este numărul de comparații, vizibil mai ales la algoritmi quadratici. Al doilea cost este numărul de mutări sau schimburi de elemente, care poate deveni dominant pentru structuri contigue și pentru elemente mari. Al treilea cost este accesul la memorie: o traversare secvențială a unui bloc compact este mult mai eficientă decât urmărirea unor pointeri dispersați în memorie.

Pentru algoritmi recursivi apare și costul apelurilor recursive, iar pentru algoritmi paraleli apare costul sincronizării. Din acest motiv, rezultatul experimental nu trebuie citit ca o simplă validare a formulei asimptotice, ci ca o sumă de factori care acționează simultan. Această perspectivă explică de ce doi algoritmi din aceeași clasă, de exemplu QuickSort și Merge Sort, pot avea timpi foarte diferiți pe același input.

7.5. Reproductibilitatea experimentului

Un aspect important al metodologiei este posibilitatea de reproducere a experimentului. Pentru aceasta, seturile de date, implementările și rapoartele brute trebuie păstrate împreună. Rapoartele brute nu sunt simple fișiere auxiliare, ci sursa primară a concluziilor. Ele permit verificarea valorilor, refacerea graficelor și compararea unor versiuni viitoare ale codului.

Într-o variantă extinsă a cercetării, fiecare rulare ar putea fi repetată de mai multe ori, iar rezultatul raportat ar putea fi media sau mediana timpilor obținuți. De asemenea, s-ar putea calcula abaterea standard pentru a estima stabilitatea măsurătorilor. În lucrarea de față, accentul este pus pe comparația relativă și pe diferențele mari de ordin de mărime, care rămân vizibile chiar și fără o analiză statistică avansată.

8. Rezultate experimentale și interpretare

8.1. Algoritmi de complexitate $O(n^2)$

Bubble Sort și Selection Sort confirmă degradarea rapidă a performanței odată cu creșterea lui n . Pentru $n = 10^5$, timpii ajung deja în zona zecilor de secunde, iar pentru valori precum 10^6 , 10^7 sau 10^8 , costurile devin impracticabile. Binary Insertion Sort reduce numărul de comparații prin căutare binară, dar deplasarea elementelor păstrează costul total în $O(n^2)$.

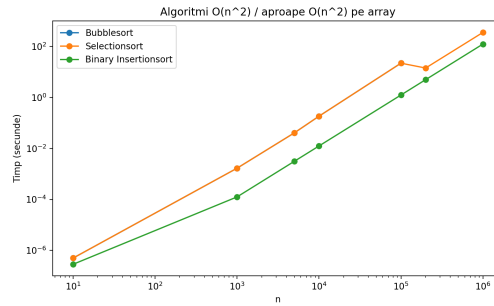


Figura 1: Evoluția timpului pentru algoritmi de complexitate $O(n^2)$, în scară logaritmică.

Rezultatul este în acord cu literatura: algoritmii quadratici sunt utili didactic și pot fi acceptabili pe volume mici, dar nu sunt potriviți pentru date mari [2, 5]. Din punct de vedere practic, aceste metode trebuie folosite cu precauție și doar atunci când dimensiunea datelor este strict limitată sau când scopul este unul educațional.

8.2. Algoritmi de complexitate $O(n \log n)$

Merge Sort, QuickSort și IntroSort confirmă superioritatea algoritmilor $O(n \log n)$. Pentru volume unde algoritmi quadratici devin inutilizabili, acești algoritmi rămân în zona secundelor sau zecilor de secunde, în funcție de implementare și structură.

În măsurătorile experimentale proprii, pentru $n = 10^8$, QuickSort pe **vector** este aproximativ 4.10 secunde, iar pe **array** aproximativ 3.85 secunde. Merge Sort clasic este mai lent, ajungând la aproximativ 47.52 secunde pe **vector** și 17.29 secunde pe **array**. IntroSort manual are aproximativ 26.77 secunde pe **vector** și 14.17 secunde pe **array**.

Diferența dintre aceste rezultate arată că notația $O(n \log n)$ nu este suficientă pentru o concluzie finală. Algoritmii au aceeași clasă medie de complexitate, dar costurile constante, accesul la memorie și strategia de implementare pot schimba masiv timpul final.

8.3. Influența structurii de date

O regularitate clară este avantajul **array**-ului. Explicația principală este localitatea în memorie: elementele sunt stocate compact, iar procesorul poate accesa eficient blocuri continue de date. Lista simplu înlănțuită pierde frecvent deoarece fiecare acces presupune traversare și dereferențiere de pointeri.

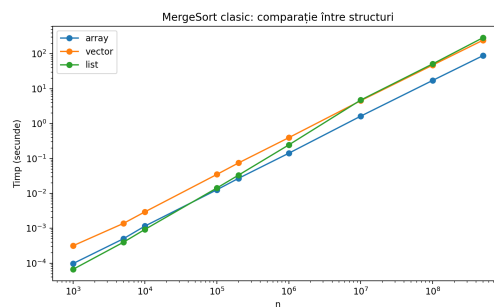


Figura 2: Merge Sort clasic: comparație între **array**, **vector** și listă simplu înlănțuită.

Pentru $n = 10^7$, Merge Sort pe `array` are aproximativ 1.64 secunde, iar pe listă aproximativ 4.73 secunde. Pentru $n = 10^8$, `array`-ul rămâne la aproximativ 17.29 secunde, iar lista depășește 51.40 secunde. Deși Merge Sort este natural pentru liste din punct de vedere conceptual, implementarea pe date contigue rămâne mai eficientă în practică.

8.4. Merge Sort paralel și QuickSort

Merge Sort paralel pe 16 thread-uri nu a depășit QuickSort-ul cu pivot random în implementarea testată. Pentru $n = 10^8$, Merge Sort paralel pe `vector` ajunge la aproximativ 14.01 secunde, în timp ce QuickSort rămâne în jur de 4.10 secunde. Pentru $n = 5 \cdot 10^8$, Merge Sort paralel este aproximativ 70.04 secunde, iar QuickSort aproximativ 23.50 secunde.

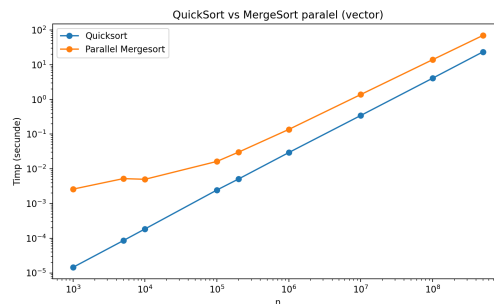


Figura 3: Comparație între QuickSort și Merge Sort paralel pe `vector`.

Explicația constă în costurile de creare și sincronizare a thread-urilor, traficul suplimentar în memorie și copierea datelor. QuickSort este in-place, iar pivotul random reduce probabilitatea unor cazuri nefavorabile [4]. Prin urmare, paralelizarea trebuie evaluată experimental, nu presupusă automat ca soluție mai bună.

8.5. IntroSort manual și `std::sort`

Deși `std::sort` este asociat în practică cu strategii de tip introsort, implementarea manuală testată a fost mai lentă. Această diferență nu arată că IntroSort este slab, ci că implementările standard sunt puternic optimizate. Musser introduce ideea introsortului ca algoritm hibrid, iar bibliotecile moderne rafinează pragurile, partiționarea și managementul recursiei [6, 3].

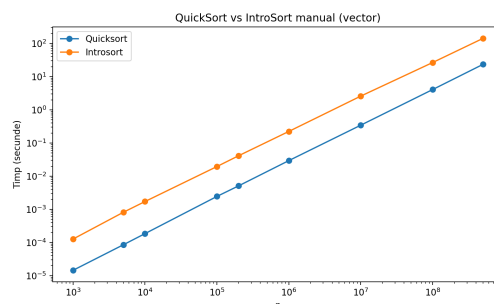


Figura 4: QuickSort vs IntroSort manual pe `vector`.

Prin urmare, performanța unui algoritm nu depinde doar de ideea algoritmică, ci și de implementarea concretă. O implementare manuală corectă poate rămâne semnificativ în urma unei implementări industriale, testate și optimizate ani de zile.

8.6. Sensibilitatea la distribuția datelor

Distribuția inputului influențează semnificativ timpul de execuție. Datele sortate, invers sortate, aproape sortate, egale sau cu multe duplicate pot modifica numărul de comparații, schimburi, partiționări și copieri. Din acest motiv, testarea doar pe date random nu este suficientă pentru o analiză completă.

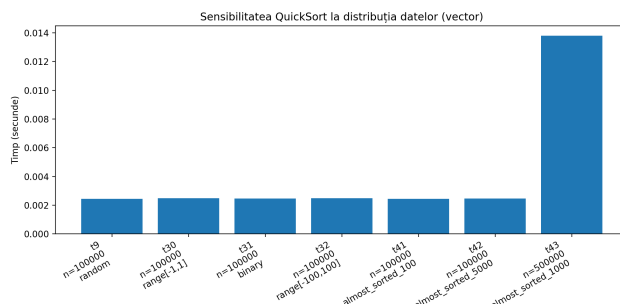


Figura 5: Sensibilitatea QuickSort la distribuția datelor pentru teste reprezentative.

În practică, algoritmul potrivit trebuie ales în funcție de natura probabilă a datelor. Dacă inputul este aproape sortat, un algoritm adaptiv poate fi avantajos. Dacă există multe duplicate, strategia de partiționare devine foarte importantă. Dacă datele sunt mari și stocate contiguu, localitatea în memorie devine un factor decisiv.

8.7. Interpretare sintetică a rezultatelor

Rezultatele pot fi sintetizate prin trei observații. Prima observație este că diferența dintre $O(n^2)$ și $O(n \log n)$ nu este doar teoretică, ci vizibilă imediat în practică. Pentru inputuri mari, algoritmii quadratici nu mai sunt opțiuni reale, indiferent cât de simplu sunt de implementat.

A doua observație este că structura de date poate schimba radical timpul de execuție. Datele stocate contiguu permit procesorului să exploateze cache-ul, în timp ce listele simplu înlanțuite produc acces indirect și localitate slabă. Această diferență explică de ce, în multe cazuri, `array`-ul a obținut cele mai bune rezultate.

A treia observație este că optimizarea implementării contează aproape la fel de mult ca alegerea algoritmului. O implementare manuală de IntroSort poate fi corectă, dar totuși mult mai lentă decât `std::sort`. În practică, algoritmul ales trebuie analizat împreună cu detaliile concrete ale codului.

Factor analizat	Effect observat	Consecință practică
Complexitatea algoritmului	separă metodele lente de cele scalabile	algoritmii $O(n^2)$ trebuie evitați pe volume mari
Structura de date	influențează localitatea în memorie	datele contigue sunt preferabile pentru performanță
Distribuția inputului	modifică numărul de comparații și partiționări	testarea trebuie făcută pe mai multe tipuri de date
Implementarea concretă	poate schimba ordinul practic al timpilor	bibliotecile standard sunt etaloane importante
Paralelizarea	introduce overhead suplimentar	nu trebuie considerată automat mai rapidă

Tabela 3: Sinteza factorilor care influențează performanța practică.

9. Comparație cu literatura de specialitate

Rezultatele obținute sunt în acord cu literatura de specialitate în privința diferenței dintre algoritmii $O(n^2)$ și cei $O(n \log n)$. Manualele clasice arată că Bubble Sort, Selection Sort și Insertion Sort au cost quadratic în cazurile generale, ceea ce limitează utilizarea lor practică la dimensiuni mici ale inputului [2, 5, 7]. Experimentele confirmă această limitare.

În cazul QuickSort, rezultatele sunt compatibile cu observațiile din lucrarea lui Hoare [4]: metoda divide et impera poate produce performanțe foarte bune în practică. Alegerea random a pivotului reduce probabilitatea cazurilor patologice și explică stabilitatea observată în testele mari.

În cazul Merge Sort, literatura evidențiază robustețea și stabilitatea algoritmului, dar și costul de memorie auxiliară [5, 2]. Rezultatele proprii susțin această observație: Merge Sort este predictibil, însă poate fi penalizat de copieri și trafic suplimentar în memorie.

În cazul IntroSort, rezultatele trebuie interpretate cu atenție. Musser [6] propune un algoritm hibrid care combină avantajele QuickSort cu garanțiile HeapSort. Implementarea manuală testată este inferioară variantelor standard optimizate, ceea ce confirmă observația lui Bentley și McIlroy că detaliile de implementare pot decide performanța finală [1].

Contribuția lucrării constă în confirmarea experimentală a unor rezultate teoretice cunoscute și în evidențierea unor factori practici: structura de date, localitatea în memorie, distribuția inputului și calitatea implementării. Lucrarea nu contrazice literatura, ci o completează printr-o perspectivă aplicată, bazată pe măsurători proprii.

10. Limite ale cercetării

Lucrarea are câteva limite care trebuie menționate pentru o interpretare corectă a rezultatelor. În primul rând, testele au fost efectuate pe o singură configurație hardware și software. Prin urmare, valorile absolute ale timpilor pot fi diferite pe alte procesoare, alte sisteme de operare sau alte compilatoare.

În al doilea rând, implementările manuale pot conține decizii de cod care influențează rezultatul. De exemplu, pragurile folosite în IntroSort sau modul de creare a thread-urilor în Merge Sort paralel pot modifica performanța. Din acest motiv, comparația cu `std::sort` trebuie

înțeleasă ca o comparație între o implementare proprie și o implementare standard foarte optimizată.

În al treilea rând, măsurarea timpului de execuție poate fi influențată de procese de sistem, încărcarea procesorului și particularitățile mediului WSL. Totuși, deoarece toate testele au fost realizate în aceleași condiții, comparațiile interne rămân relevante.

11. Direcții viitoare de cercetare

Rezultatele obținute deschid mai multe direcții de cercetare:

1. **Analiza distribuției datelor:** gruparea formală a testelor pe clase de input și construirea unor profile de sensibilitate pentru fiecare algoritm.
2. **Studiul localității în memorie:** corelarea timpilor de execuție cu rata de cache miss, traficul în memorie și costurile de dereferențiere.
3. **Algoritmi hibridi:** calibrarea pragurilor pentru trecerea de la QuickSort la HeapSort sau Insertion Sort în implementări de tip IntroSort.
4. **Strategii de pivot:** compararea pivotului random cu median-of-three, primul element, ultimul element sau alte euristici.
5. **Paralelizare:** determinarea pragului de la care costul thread-urilor devine justificat de câștigul de performanță.
6. **Normalizarea datelor:** testarea transformării valorilor reale sau caracterelor în reprezentări întregi, cu atenție la overflow și pierdere de informație.

Aceste direcții nu schimbă concluzia de bază, ci o extind: performanța reală a sortării este rezultatul interacțiunii dintre algoritm, date, memorie, implementare și platformă.

12. Transpunerea rezultatelor în forma lucrării

Forma finală a lucrării urmează logica unei cercetări științifice aplicate. Introducerea formulează problema, justifică tema și stabilește motivația. Partea principală conține cadrul formal, descrierea soluțiilor existente, metodologia, modelarea implementării și experimentul. Încheierea include comparația cu literatura de specialitate, limitele cercetării, direcțiile viitoare și concluziile.

Această structură este importantă deoarece cititorul nu trebuie să primească doar o listă de timpi, ci o explicație coerentă a modului în care acei timpi au fost obținuți și interpretați. O lucrare despre algoritmi trebuie să lege teoria de implementare și implementarea de rezultat. Fără această legătură, graficele ar rămâne simple observații izolate.

În forma actuală, rolul părții experimentale este de a confirma, nu de a înlocui, analiza teoretică. Literatura stabilește cadrul general, iar experimentul arată cum se manifestă acel cadru într-un mediu concret. Această relație dintre teorie și măsurare reprezintă nucleul lucrării.

13. Concluzii finale

Lucrarea a analizat experimental mai mulți algoritmi de sortare, urmărind diferența dintre comportamentul teoretic și performanța practică. Rezultatele arată că algoritmi $O(n^2)$ sunt improprii pentru volume mari de date, în timp ce algoritmi $O(n \log n)$ reprezintă alegerea corectă pentru seturi mari.

`Array`-ul a fost, în general, cea mai eficientă structură, datorită stocării contigue și localității bune în memorie. Lista simplu înlănțuită a fost penalizată de accesul secvențial și de costurile pointerilor. De asemenea, paralelizarea nu a garantat superioritate practică, deoarece overhead-ul poate anula beneficiul execuției concurente.

O concluzie importantă este că implementările standard, precum `std::sort`, trebuie tratate ca etalon practic, nu doar ca referință teoretică. Diferența dintre IntroSort manual și implementările standard arată că performanța depinde decisiv de rafinarea codului.

În ansamblu, alegerea algoritmului optim nu depinde doar de complexitatea teoretică, ci de interacțiunea dintre algoritm, structură de date, distribuția inputului, reprezentarea tipului de date, arhitectura de memorie și calitatea implementării. Aceasta este contribuția principală a lucrării: trecerea de la o comparație pur teoretică la o analiză experimentală a comportamentului real al algoritmilor de sortare.

Bibliografie

- [1] Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] cppreference.com. `std::sort`. <https://en.cppreference.com/w/cpp/algorithm/sort>.
- [4] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [5] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998.
- [6] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.
- [7] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4 edition, 2011.
- [8] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.